Models

- doctor

```
import { Hospital } from "./hospital.model";
import { Patient } from "./patient.model";

export interface Doctor {
    doctorId: string;
    name?: string | null;
    specialization?: string | null;
    hospitalId: string;
    hospital?: Hospital | null;
    patients?: Patient[] | null;
}
```

- hospital

```
import { Doctor } from "./doctor.model";
import { Patient } from "./patient.model";

export interface Hospital {
    hospitalId: string;
    name?: string | null;
    doctors?: Doctor[] | null;
    patients?: Patient[] | null;
}
```

-patient

```
import { Doctor } from "./doctor.model";
import { Hospital } from "./hospital.model";

export interface Patient {
    patientId: string;
    name?: string | null;
    hospitalId: string;
    hospital?: Hospital | null;
    doctors?: Doctor[] | null;
}
```

- user and role

```
export interface User {
    userId?: string | null;
    userName?: string | null;
    email?: string | null;
    password?: string | null; // used only for
login/register
    passwordHash?: string | null;
    role:Role
}

export interface Role{
   roleId?: string | null;
    roleName?: string | null;
}
```

Services

- auth.services.ts

```
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable, tap } from 'rxjs';
import { User } from './models/user.model';

@Injectable({
  providedIn: 'root'
})
export class AuthService {

  private base = "https://localhost:7071/api/Token";

  constructor(private http: HttpClient) { }
  login(username: string, password: string):
Observable<{
    token: string; username: string; role: string;
  }> {
    return this.http
      .post<{ token: string; username: string; role:
string }>(`${this.base}/login`, {
        username,
        password
      })
      .pipe(
        tap((resp) => {
          if (resp?.token) {
            localStorage.setItem('jwt_token',
resp.token);
            localStorage.setItem('current_user',
JSON.stringify({
              username: resp.username,
```

```
              role: resp.role
            }));
          }
        })
      );
  }
  register(user: User): Observable<User> {
    return
this.http.post<User>(`${this.base}/register`, user);
  }
  logout(): void {
    localStorage.removeItem('jwt_token');
    localStorage.removeItem('current_user');
  }
  getToken(): string | null {
    return localStorage.getItem('jwt_token');
  }
  getCurrentUser(): User | null {
    const v = localStorage.getItem('current_user');
    return v ? JSON.parse(v) : null;
  }
  isLoggedIn(): boolean {
    return !!this.getToken();
  }

  getCurrentUserRole(): string | null {
  const user = this.getCurrentUser();
  return user?.role?.roleName ?? null;
}

isAdmin(): boolean {
  return this.getCurrentUserRole() === 'Admin';
}
```

```
isPatient(): boolean {
  return this.getCurrentUserRole() === 'Patient';
}

isUser(): boolean {
  return this.getCurrentUserRole() === 'User';
}

}
```

- doctor.services.ts

```
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Doctor } from './models/doctor.model';
import { Observable } from 'rxjs';
import { Hospital } from './models/hospital.model';

@Injectable({
  providedIn: 'root'
})
export class DoctorserService {

  private base = "https://localhost:7071/api";

  constructor(private http: HttpClient) { }
  getAll(): Observable<Doctor[]> {
    return
this.http.get<Doctor[]>(`${this.base}/Doctors`);
  }
  getById(id: string): Observable<Doctor> {
    return
this.http.get<Doctor>(`${this.base}/${id}`);
  }
```

```
  getHospitals(): Observable<Hospital[]> {
    return
this.http.get<Hospital[]>(`${this.base}/Hospitals`);
  }
  add(doc: Partial<Doctor>): Observable<Doctor> {
    return
this.http.post<Doctor>(`${this.base}/Doctors`, doc);
  }
  update(doc: Doctor): Observable<Doctor> {

    return
this.http.put<Doctor>(`${this.base}/${doc.doctorId}`,
doc);
  }
  delete(id: string): Observable<any> {
    return this.http.delete(`${this.base}/${id}`);
  }
}
```

- hospital.services.ts

- patient.services.ts

- user.services.ts

```
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { User } from './models/user.model';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class UserserService {
```

```
  private base = "https://localhost:7071/api/Users";

  constructor(private http: HttpClient) { }
  getAll(): Observable<User[]> {
    return this.http.get<User[]>(this.base);
  }
  getById(id: string): Observable<User> {
    return this.http.get<User>(`${this.base}/${id}`);
  }
  add(u: Partial<User>): Observable<User> {
    return this.http.post<User>(this.base, u);
  }
  update(u: User): Observable<User> {
    return
this.http.put<User>(`${this.base}/${u.userId}`, u);
  }
  delete(id: string): Observable<any> {
    return this.http.delete(`${this.base}/${id}`);
  }

}
```

Interceptors

- auth.interceptor.ts

```
import { HttpInterceptorFn } from
'@angular/common/http';

export const authInterceptor: HttpInterceptorFn =
(req, next) => {
  const token = localStorage.getItem('jwt_token');

  if (token) {
```

```
    req = req.clone({
      setHeaders: {
        Authorization: `Bearer ${token}`
      }
    });
  }

  return next(req);
};
```

Guards

- auth.guard.ts

```
import { Injectable } from '@angular/core';
import { CanActivate, Router } from
'@angular/router';
import { AuthService } from '../auth.service';

@Injectable({ providedIn: 'root' })
export class authGuard implements CanActivate {
  constructor(private auth: AuthService, private
router: Router) { }
  canActivate(): boolean {
    if (this.auth.isLoggedIn()) return true;
    console.log('Token:', this.auth.getToken());
    this.router.navigate(['/login']);

    return false;
  }
}
```

- role.guard.ts

```
import { Injectable } from '@angular/core';
import { ActivatedRouteSnapshot, CanActivate, Router
} from '@angular/router';
import { AuthService } from '../auth.service';

@Injectable({ providedIn: 'root' })
export class RoleGuard implements CanActivate {
  constructor(private auth: AuthService, private
router: Router) { }
  canActivate(route: ActivatedRouteSnapshot): boolean
{
    const expectedRoles: string[] =
route.data['roles'];
    const user = this.auth.getCurrentUser();
    if (!user || !user.role || !expectedRoles ||
expectedRoles.length === 0) {

      this.router.navigate(['/login']);
      return false;
    }
    if (expectedRoles.includes(user.role.roleName!))
{

      return true;
    }
        console.log('Current User:',
this.auth.getCurrentUser());
    console.log('Expected Roles:',
route.data['roles']);
    // not allowed
    this.router.navigate(['/']);
```

```
    return false;
    }
}
```

App.routes.ts

- set the paths

```
    { path: '', redirectTo: 'doctors', pathMatch: 'full' },
    { path: 'login', component: LoginComponent },
    { path: 'register', component: RegisterComponent },
    { path: 'users', component: UserComponent},
    // Doctors - admin and doctor roles
    {
        path: 'doctors',
        component: DoctorcomComponent,
        canActivate: [authGuard]
    },
    {path:'doctors/new',component:DoctorpostComponent,canActivate:
[authGuard,RoleGuard],
        data:{roles:['Admin']}
    }
```

app.component.ts

```
export class AppComponent {


  title = 'day6ang';

  constructor(public auth: AuthService) {}

  logout() {
    this.auth.logout();
  }
}
```

app.compoent.html

```
<nav class="navbar navbar-expand-lg navbar-light bg-
light">
  <div class="container-fluid">
    <a class="navbar-brand" routerLink="/">Hospital
App</a>
    <div class="collapse navbar-collapse">
      <ul class="navbar-nav me-auto mb-2 mb-lg-0">
        <li class="nav-item"><a class="nav-link"
routerLink="/doctors">Doctors</ a>
        </li>
        <li class="nav-item"><a class="nav-link"
routerLink="/hospitals">Hospitals</a></li>
        <li class="nav-item"><a class="nav-link"
routerLink="/patients">Patients</a></li>
        <li class="nav-item"><a class="nav-link"
routerLink="/users">Users</a></li>
      </ul>
      <ul class="navbar-nav">
        <li *ngIf="!auth.isLoggedIn()" class="nav-
item"><a class="nav-link"
routerLink="/login">Login</a></li>
        <li *ngIf="!auth.isLoggedIn()" class="nav-
item"><a class="nav-link"
routerLink="/register">Register</a></li>

        <li *ngIf="auth.isLoggedIn()" class="nav-item
dropdown">
          <a class="nav-link dropdown-toggle"
role="button" data-bstoggle="dropdown">{{
auth.getCurrentUser()?.userName
```

```
            }}</a>
          <ul class="dropdown-menu dropdown-menu-
end">
            <li><span class="dropdown-item">Role:
                {{
auth.getCurrentUser()?.role?.roleName }}</span></li>
            <li><button class="dropdown-item"
(click)="logout()">Logout</ button>
            </li>
          </ul>
        </li>
      </ul>
    </div>
  </div>
</nav>
<div class="container mt-3">
  <router-outlet></router-outlet>
</div>
```

app.config.ts

```
import { ApplicationConfig,
provideZoneChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';

import { routes } from './app.routes';
import { provideClientHydration, withEventReplay }
from '@angular/platform-browser';
import { provideHttpClient, withFetch,
withInterceptors } from '@angular/common/http';
import { authInterceptor } from
'./interceptors/auth.interceptor';

export const appConfig: ApplicationConfig = {
```

```
  providers: [provideZoneChangeDetection({
eventCoalescing: true }),
    provideHttpClient(withFetch(),
withInterceptors([authInterceptor])),provideRouter(ro
utes), provideClientHydration(withEventReplay())]
};
```

Components - Auth

login.component.ts

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, FormsModule,
ReactiveFormsModule, Validators } from
'@angular/forms';
import { AuthService } from '../auth.service';
import { Router } from '@angular/router';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-login',
  imports:
[FormsModule,CommonModule,ReactiveFormsModule],
  templateUrl: './login.component.html',
  styleUrl: './login.component.css'
})
export class LoginComponent {
  form: FormGroup;
  error = '';
  constructor(private fb: FormBuilder, private auth:
AuthService, private router: Router) {
    this.form = this.fb.group({
      username: ['', Validators.required], password:
        ['', Validators.required]
    });
```

```
  }
  submit() {
    if (this.form.invalid) return;

    const { username, password } = this.form.value;
    this.auth.login(username, password).subscribe({
      next: () => this.router.navigate(['/']),
      error: (err) => (this.error =
err?.error?.message || 'Login failed')
    });
  }
}
```

login.component.html

```
<div class="row justify-content-center">
    <div class="col-md-6">
        <h2>Login</h2>
        <form [formGroup]="form"
(ngSubmit)="submit()">
            <div class="mb-3">
                <label>Username</label>
                <input formControlName="username"
class="form-control" />
            </div>
            <div class="mb-3">
                <label>Password</label>
                <input formControlName="password"
type="password" class="form-control" />
            </div>
            <div *ngIf="error" class="alert alert-
danger">{{ error }}</div>
            <button class="btn btn-primary"
type="submit">Login</button>
```

```
        </form>
    </div>
</div>
```

register.component.ts

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, FormsModule,
ReactiveFormsModule, Validators } from
'@angular/forms';
import { AuthService } from '../auth.service';
import { Router } from '@angular/router';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-register',
  imports:
[FormsModule,CommonModule,ReactiveFormsModule],
  templateUrl: './register.component.html',
  styleUrl: './register.component.css'
})
export class RegisterComponent {
  form: FormGroup;
  error = '';
  constructor(private fb: FormBuilder, private auth:
AuthService, private router: Router) {

    this.form = this.fb.group({
      userName: ['', Validators.required],
      email: ['', [Validators.required,
Validators.email]],
      password: ['', Validators.required],
      roleId: ['', Validators.required]
    });
```

```
  }
  submit() {
    if (this.form.invalid) return;
    const payload = {
      userName: this.form.value.userName,
      email: this.form.value.email,
      password: this.form.value.password,
      roleId: this.form.value.roleId
    };
    this.auth.register(payload as any).subscribe({
      next: () => this.router.navigate(['/login']),
      error: (err) => (this.error =
err?.error?.message || 'Registrationfailed')
    });
  }

}
```

register.component.html

```
<div class="row justify-content-center">
    <div class="col-md-6">
        <h2>Register</h2>
        <form [formGroup]="form"
(ngSubmit)="submit()">
            <div class="mb-3">
                <label>Username</label>
                <input formControlName="userName"
class="form-control" />
            </div>
            <div class="mb-3">
                <label>Email</label>
                <input formControlName="email"
class="form-control" />
```

```
            </div>
            <div class="mb-3">
                <label>Password</label>
                <input formControlName="password"
type="password" class="form-control" />
            </div>
            <div class="mb-3">
                <label>Role ID (ex: R001 for Admin,
R002 for Doctor, R003 for Patient)</label>
                    <input formControlName="roleId"
class="form-control" />
            </div>
            <div *ngIf="error" class="alert alert-
danger">{{ error }}</div>
            <button class="btn btn-primary"
type="submit">Register</button>
        </form>
    </div>
</div>
```

Components - Doctors

doctors.component.ts

```
import { Component, OnInit } from '@angular/core';
import { DoctorserService } from
'../doctorser.service';
import { Doctor } from '../models/doctor.model';
import { Router } from '@angular/router';
import { CommonModule } from '@angular/common';
import { AuthService } from '../auth.service';
@Component({
  selector: 'app-doctorcom',
  standalone:true,
  imports: [CommonModule],
```

```
  templateUrl: './doctorcom.component.html',
  styleUrl: './doctorcom.component.css'
})
export class DoctorcomComponent implements OnInit{
  doctors: Doctor[] = [];
  loading = false;
  error = '';
  constructor(private service:
DoctorserService,public auth:AuthService,private
router: Router) { }
  ngOnInit() {
    this.load();
  }
  load() {
    this.loading = true;
    this.service.getAll().subscribe({
      next: (d) => ((this.doctors = d),
        (this.loading = false)), error: () =>
((this.error = 'Could not load doctors'),
          (this.loading = false))
    });
  }
  add() {
    console.log("welcome");
    this.router.navigate(['/doctors/new']);
  }
  edit(id: string) {
    this.router.navigate(['/doctors/edit', id]);
  }

  delete(id: string) {
    if (!confirm('Delete doctor?')) return;
    this.service.delete(id).subscribe({
      next: () => this.load(), error: () =>
```

```
        alert('Delete failed')
    });
  }

}
```

doctors.component.html

```
<p>doctorcom works!</p>
<div class="d-flex justify-content-between align-
items-center mb-2">
    <h3>Doctors</h3>
    <button class="btn btn-success"
*ngIf="auth.isAdmin()" (click)="add()">New</button>
</div>
<div *ngIf="loading">Loading...</div>
<div *ngIf="error" class="alert alert-danger">{{
error }}</div>
<table class="table table-striped" *ngIf="!loading &&
doctors?.length">
    <thead>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Specialization</th>
            <th>Hospital</th>
            <th>
                </th>
        </tr>
    </thead>
    <tbody>
        <tr *ngFor="let d of doctors">
            <td>{{ d.doctorId }}</td>
            <td>{{ d.name }}</td>
```

```
            <td>{{ d.specialization }}</td>
            <td>{{ d.hospital?.name }}</td>
            <td>
                <button class="btn btn-sm btn-primary
me-1" *ngIf="auth.isAdmin()"
(click)="edit(d.doctorId)">Edit</button>
                <button class="btn btn-sm btn-
danger"  *ngIf="auth.isAdmin()"
(click)="delete(d.doctorId)">Delete</button>
            </td>
        </tr>
    </tbody>
</table>
<div *ngIf="!loading && (!doctors || doctors.length
=== 0)" class="alert alertinfo">No doctors
found.</div>
```

--get of doctors with authorization

doctors-form.component.ts

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, ReactiveFormsModule,
Validators } from '@angular/forms';
import { Hospital } from '../models/hospital.model';
import { ActivatedRoute, Router, RouterModule } from
'@angular/router';
import { DoctorserService } from
'../doctorser.service';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-doctorpost',
```

```
  standalone:true,
  imports:
[ReactiveFormsModule,RouterModule,CommonModule],
  templateUrl: './doctorpost.component.html',
  styleUrl: './doctorpost.component.css'
})
export class DoctorpostComponent {
  form: FormGroup;
  id?: string;
  isEdit = false;
  hospitals: Hospital[] = [];
  constructor(private fb: FormBuilder,private route:
ActivatedRoute,private router: Router,private svc:
DoctorserService)
  {
    this.form = this.fb.group({
      doctorId: [''], name: [''],
        Validators.required], specialization: [''],
hospitalId: ['']
    });
  }
  ngOnInit() {
    this.svc.getHospitals().subscribe((h) =>
(this.hospitals = h));
    this.id = this.route.snapshot.paramMap.get('id')
|| undefined;
    if (this.id) {
      this.isEdit = true;
      this.svc.getById(this.id).subscribe((d) =>
this.form.patchValue(d));
    }
  }
  save() {
    if (this.form.invalid) return;
```

```
    const val = this.form.value;
    if (this.isEdit) {
      this.svc.update(val).subscribe(() =>
this.router.navigate(['/doctors']));
    } else {
      const payload = {
        name: val.name, specialization:
val.specialization,
        hospitalId: val.hospitalId
      };
      this.svc.add(payload).subscribe(() =>
this.router.navigate(['/doctors']));
    }
  }
}
```

doctors-form.component.html

--add,edit and delete of doctor with role guard

```
<h3>{{ isEdit ? 'Edit Doctor' : 'New Doctor' }}</h3>
<form [formGroup]="form" (ngSubmit)="save()">
    <div class="mb-3">
        <label>Name</label>
        <input formControlName="name" class="form-
control" />
    </div>
    <div class="mb-3">
        <label>Specialization</label>
        <input formControlName="specialization"
class="form-control" />
```

```
    </div>
    <div class="mb-3">
        <label>Hospital</label>
        <select formControlName="hospitalId"
class="form-control">
            <option value="">-- Select --</option>
            <option *ngFor="let h of hospitals"
[value]="h.hospitalId">{{ h.name }}</ option>
        </select>
    </div>
    <button class="btn btn-primary" type="submit">{{
isEdit ? 'Update' :
        'Create' }}</button>
    <a class="btn btn-secondary ms-2"
routerLink="/doctors">Cancel</a>
</form>
```